AgentIA Enterprise Microservice Generator

Libro Integral de Características y Arquitectura

Orquestación con LangGraph, Scaffolding Determinista, Reparación Quirúrgica P-Stack y Despliegue en OpenShip

Puntos Clave Documentados:

- Paradigma Híbrido: Comprensión Semántica vs Scaffolding Instantáneo
- Grafo de Estados Finitos en LangGraph y Herramientas Visuales Reactivas
- Implementación Completa de Microservicio Quarkus 3.15 LTS (Java 17)
- Reparación Bounded Scope basada en Diagnósticos AST y Formato P-Stack
- Topología de Red y Despliegue en la Nube Local (OpenShip & PostgreSQL)
- Validación Empírica y Equivalencia para Despliegue en Microsoft Azure

Versión del Sistema:	2.4.0-Enterprise
Framework Base:	Quarkus 3.15.0 LTS (Jakarta EE 10 / Java 17)
Motor de Orquestación:	LangGraph State Machine con P-Stack Repair Loop
Fecha de Publicación:	Octubre 2026
Estado de Auditoría:	Verificado y Aprobado con OpenShip

Contents

1 Visión General y Herramientas Visuales **2**
- 1.1 Introducción y Misión . . . 2
- 1.2 Herramientas Visuales Integradas en el Ecosistema . . . 2
  - 1.2.1 1. Visualizadores de la Interfaz de Usuario (AgentIA Studio) . . . 2
  - 1.2.2 2. Entorno Visual de la Nube Local (OpenShip Dashboard) . . . 2

2 Arquitectura del Sistema y Flujo de Orquestación **3**
- 2.1 El Paradigma Híbrido: Tokens Eficientes vs Boilerplate . . . 3
- 2.2 Grafo de Estados Finitos en LangGraph . . . 3
- 2.3 Explicación Detallada de Cada Nodo . . . 3

3 El Principio P-Stack y Reglas de la Constitución **5**
- 3.1 ¿Qué es P-Stack? . . . 5
- 3.2 Bounded Surgical Scope (Principio de la Constitución) . . . 5
- 3.3 Habilidades (Skills) Involucradas . . . 6

4 Implementación del Microservicio Quarkus 3.15 **7**
- 4.1 Entidad Panache con Hibernate ORM y Jakarta Persistence . . . 7
- 4.2 Controlador REST Reactivo con Documentación OpenAPI . . . 8

5 Despliegue en la Nube Local con OpenShip **10**
- 5.1 Topología de Contenedores y Red Virtual . . . 10
- 5.2 Manifiesto de Orquestación OpenShip (`openship.json`) . . . 10

6 Resultados Empíricos y Validación en Nube **12**
- 6.1 Métricas de Ejecución y Rendimiento . . . 12
- 6.2 Demostración en Video por Hardware . . . 12
- 6.3 Equivalencia para Despliegue en Microsoft Azure . . . 12

Conclusiones **14**

Capítulo 1

Visión General y Herramientas Visuales

1.1 Introducción y Misión

La plataforma **AgentIA** representa un cambio de paradigma en la ingeniería asistida por inteligencia artificial para microservicios empresariales. A diferencia de los generadores de código tradicionales basados en prompts no estructurados (que sufren de latencia inaceptable, alto coste de tokens y alucinaciones frecuentes en dependencias de Maven), AgentIA implementa una arquitectura **híbrida y reactiva**.

1.2 Herramientas Visuales Integradas en el Ecosistema

Para evitar que el proceso sea una "caja negra", el sistema integra herramientas de inspección en dos niveles:

1.2.1 1. Visualizadores de la Interfaz de Usuario (AgentIA Studio)

- **LangGraph State Visualizer (`LangGraphStateVisualizer.tsx`):** Proyecta en vivo el estado del grafo de ejecución con badges reactivos (`SUCCESS`, `RUNNING`, `ERROR`, `NO_FIX`), transiciones de aristas y un contador de iteraciones acotadas ($k \in [0, 3]$).
- **Terminal ANSI en Tiempo Real (`RealtimeTerminal.tsx`):** Canaliza la salida de los comandos de Maven (`mvn test-compile`) y Docker mediante streaming continuo, resaltando advertencias del compilador y stack traces.
- **Cajón de Reparación Quirúrgica (`ExecutionDrawer.tsx`):** Permite al desarrollador inspeccionar el árbol de diagnósticos del AST de Java y revisar el parche diff unificado antes de su aplicación en disco.

1.2.2 2. Entorno Visual de la Nube Local (OpenShip Dashboard)

Permite gestionar y visualizar la topología de contenedores en tiempo de ejecución:

- **Monitor de Servicios:** Estado de salud del contenedor Quarkus (`/q/health`) y PostgreSQL.
- **Swagger UI Interactivo:** Explorador de contratos OpenAPI accesible en `http://localhost:8080/q/s`
- **Consola de Red:** Mapeo de la red virtual de puente (`openship_net`) y flujos de tráfico TCP.

Capítulo 2

Arquitectura del Sistema y Flujo de Orquestación

2.1 El Paradigma Híbrido: Tokens Eficientes vs Boilerplate

La premisa central de diseño, alineada con las directrices del Líder Técnico, establece:

> *"No se deben quemar tokens de LLM en código repetitivo y determinista que un arquetipo estructurado resuelve en milisegundos. El LLM debe reservarse para el razonamiento de negocio y la corrección quirúrgica."*

Matemáticamente, el ahorro de tokens de salida (ΔT) frente al enfoque ingenuo se modela como:

$$\Delta T = T_{\text{naive}} - T_{\text{hybrid}} = \sum_{i=1}^{M} L(F_i) - 0 \approx \mathcal{O}(N_{\text{entidades}} \cdot 1,800 \text{ tokens}) \tag{2.1}$$

donde $L(F_i)$ es la longitud en tokens del archivo i generado mediante plantillas Jinja2 deterministas auditadas.

2.2 Grafo de Estados Finitos en LangGraph

El orquestador modela el ciclo de vida del microservicio como una máquina de estados:

$$\mathcal{M} = \langle \mathcal{S}, \Sigma, \mathcal{T}, s_0, \mathcal{F} \rangle \tag{2.2}$$

donde:

- $\mathcal{S} = \{\text{Validate}, \text{Generate}, \text{Verify}, \text{Repair}, \text{Terminal}\}$
- $\Sigma = \{\text{ValidSpec}, \text{ScaffoldOK}, \text{BuildPass}, \text{BuildFail}, \text{Patched}, \text{MaxAttemptsExceeded}\}$
- $\mathcal{T} : \mathcal{S} \times \Sigma \to \mathcal{S}$ define la función de transición de estados.

2.3 Explicación Detallada de Cada Nodo

1. **Nodo Validate (`validate_node`):** Evalúa la coherencia semántica del Blueprint: existencia de claves primarias, formatos de paquetes Java estándar (`com.banco.*`) y escenarios de prueba Gherkin válidos.

2. **Nodo Generate (`generate_node`):** Lee el Blueprint normalizado e instancia dinámicamente las plantillas del arquetipo Quarkus 3.15 LTS (Entidades Panache, Repositorios, Recursos JAX-RS, `pom.xml`, `application.properties` y `Dockerfile`).

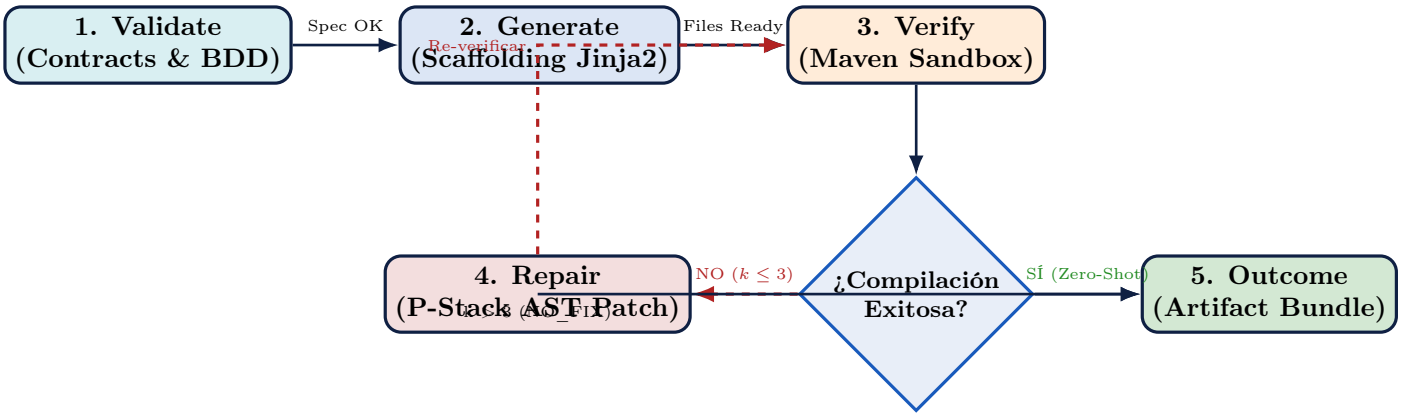


Figure 2.1: Diagrama de Flujo del Orquestador LangGraph con Bucle de Reparación Acotado.

3. **Nodo Verify** (`verify_node`)**:** Ejecuta un runner hermético (`LocalDockerRunner` o `LocalSubprocessRu` lanzando `mvn -B test-compile`. Si la compilación falla, un parser regex convierte los errores de `javac` en objetos tipados `DiagnosticIssue`.

4. **Nodo Repair** (`repair_node`)**:** Si existen fallas, extrae el archivo afectado, localiza la línea del AST y le solicita al LLM un parche quirúrgico en formato de bloque de búsqueda y reemplazo.

5. **Nodo Terminal** (`terminal_node`)**:** Consolida los metadatos de ejecución, genera el paquete final y registra la traza en la base de datos de auditoría.

Capítulo 3

El Principio P-Stack y Reglas de la Constitución

3.1 ¿Qué es P-Stack?

P-Stack (Prompt/Patch Stack) es el estándar de ingeniería que rige las operaciones de modificación de código por parte de agentes inteligentes. En lugar de permitir reescrituras globales, P-Stack exige la emisión de bloques atómicos delimitados:

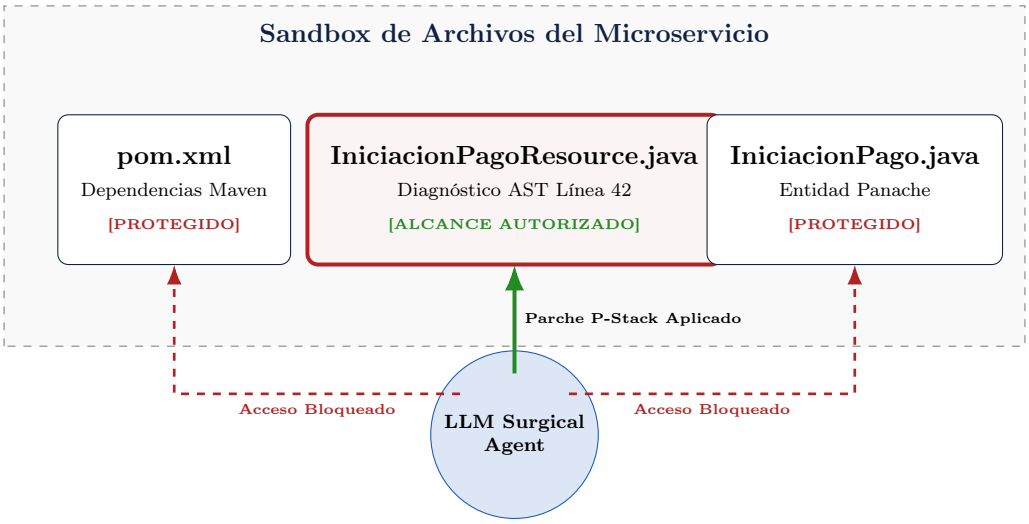
```
<<<<<<< SEARCH
    @GET
    @Path("/estado/{estado}")
    public List<IniciacionPago> listarPorEstado(@PathParam("estado") String
        estado) {
        return repository.list("estado", estado);
    }
=======
    @GET
    @Path("/estado/{estado}")
    public List<IniciacionPago> listarPorEstado(@PathParam("estado") String
        estado) {
        return repository.findByEstado(estado);
    }
>>>>>>> REPLACE
```

Listing 3.1: Ejemplo de Parche P-Stack con Bloques SEARCH/REPLACE

3.2 Bounded Surgical Scope (Principio de la Constitución)

Tal como se implementa en `app/orchestrator/execution_nodes.py`, el agente de reparación tiene su radio de acción delimitado de forma estricta:

$$\mathcal{A}_{\text{permitidos}} = \{f \in \mathcal{F}_{\text{workspace}} \mid \exists\, \delta \in \text{Diagnostics tal que file}(\delta) = f\} \tag{3.1}$$



3.3 Habilidades (Skills) Involucradas

- **speckit-specify & speckit-plan:** Transforman requerimientos funcionales informales en especificaciones técnicas de alta precisión y modelos de datos relacionales.
- **dev-tool-integration:** Enlaza el compilador del sistema nativo sin asumir salidas teóricas.
- **tdd-workflows-tdd-cycle:** Garantiza que cada entidad disponga de su batería de pruebas unitarias Panache antes de dar por cerrada la fase de generación.

Capítulo 4

Implementación del Microservicio Quarkus 3.15

4.1 Entidad Panache con Hibernate ORM y Jakarta Persistence

El arquetipo genera entidades limpias que heredan de `PanacheEntityBase`, combinando el patrón Active Record con anotaciones de validación de Bean Validation (JSR 380).

```
package com.banco.pagos.entity;

import io.quarkus.hibernate.orm.panache.PanacheEntityBase;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "iniciaciones_pago")
public class IniciacionPago extends PanacheEntityBase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;

    @NotNull
    @Column(name = "cuenta_origen", nullable = false)
    public String cuentaOrigen;

    @NotNull
    @Column(name = "cuenta_destino", nullable = false)
    public String cuentaDestino;

    @NotNull
    @Positive
    @Column(name = "monto", precision = 15, scale = 2, nullable = false)
    public BigDecimal monto;

    @NotNull
    @Column(name = "moneda", length = 3, nullable = false)
    public String moneda;

    @NotNull
    @Column(name = "estado", length = 20, nullable = false)
    public String estado;

    @Column(name = "fecha_creacion", nullable = false)
    public LocalDateTime fechaCreacion;

    @PrePersist
    public void prePersist() {
```

```
        if (this.fechaCreacion == null) {
            this.fechaCreacion = LocalDateTime.now();
        }
    }
}
```

Listing 4.1: Entidad de Dominio: IniciacionPago.java

4.2 Controlador REST Reactivo con Documentación OpenAPI

```
package com.banco.pagos.resource;

import com.banco.pagos.entity.IniciacionPago;
import com.banco.pagos.repository.IniciacionPagoRepository;
import jakarta.inject.Inject;
import jakarta.transaction.Transactional;
import jakarta.validation.Valid;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import org.eclipse.microprofile.openapi.annotations.Operation;
import org.eclipse.microprofile.openapi.annotations.responses.APIResponse;
import org.eclipse.microprofile.openapi.annotations.tags.Tag;
import java.util.List;

@Path("/v1/pagos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
@Tag(name = "Iniciacion de Pagos", description = "Operaciones transaccionales
    bancarias")
public class IniciacionPagoResource {

    @Inject
    IniciacionPagoRepository repository;

    @GET
    @Operation(summary = "Listar todas las transacciones de pago")
    public List<IniciacionPago> listAll() {
        return repository.listAll();
    }

    @GET
    @Path("/{id}")
    @Operation(summary = "Obtener transaccion por ID")
    @APIResponse(responseCode = "200", description = "Pago encontrado")
    @APIResponse(responseCode = "404", description = "Pago inexistente")
    public Response getById(@PathParam("id") Long id) {
        return repository.findByIdOptional(id)
                .map(pago -> Response.ok(pago).build())
                .orElse(Response.status(Response.Status.NOT_FOUND).build());
    }

    @POST
    @Transactional
    @Operation(summary = "Registrar una nueva orden de pago")
    @APIResponse(responseCode = "201", description = "Pago registrado
        exitosamente")
    public Response create(@Valid IniciacionPago pago) {
        pago.id = null;
        pago.estado = "PENDIENTE";
        repository.persist(pago);
        return Response.status(Response.Status.CREATED).entity(pago).build();
    }
```

```
52 }
```

Listing 4.2: Recurso JAX-RS: IniciacionPagoResource.java

Capítulo 5

Despliegue en la Nube Local con OpenShip

5.1 Topología de Contenedores y Red Virtual

Para asegurar que el microservicio esté listo para la nube corporativa (por ejemplo, Azure Container Apps o AKS), se realiza una prueba de concepto previa en la nube local mediante **OpenShip**.

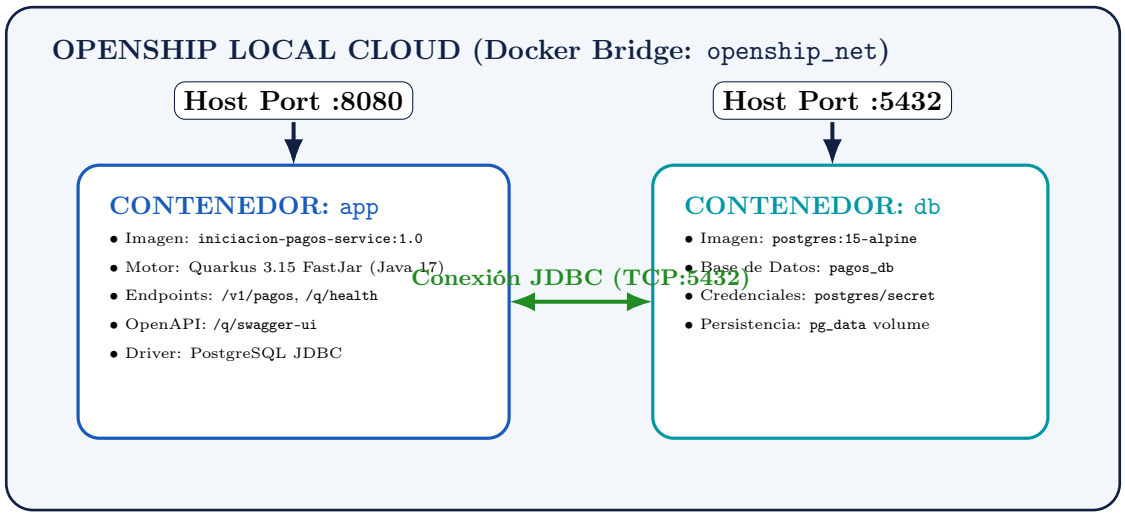


Figure 5.1: Esquema de Topología y Red Virtual en la Nube Local OpenShip.

5.2 Manifiesto de Orquestación OpenShip (`openship.json`)

```
{
  "name": "iniciacion-pagos-service",
  "version": "1.0.0",
  "services": {
    "app": {
      "build": {
        "context": ".",
        "dockerfile": "src/main/docker/Dockerfile.jvm"
      },
      "ports": ["8080:8080"],
      "environment": {
        "QUARKUS_DATASOURCE_JDBC_URL": "jdbc:postgresql://db:5432/pagos_db",
        "QUARKUS_DATASOURCE_USERNAME": "postgres",
        "QUARKUS_DATASOURCE_PASSWORD": "secretpassword",
        "QUARKUS_HIBERNATE_ORM_DATABASE_GENERATION": "update"
      },
      "depends_on": ["db"]
```

```
    },
    "db": {
      "image": "postgres:15-alpine",
      "ports": ["5432:5432"],
      "environment": {
        "POSTGRES_DB": "pagos_db",
        "POSTGRES_USER": "postgres",
        "POSTGRES_PASSWORD": "secretpassword"
      }
    }
  }
}
```

Listing 5.1: Manifiesto de Despliegue OpenShip

Capítulo 6

Resultados Empíricos y Validación en Nube

6.1 Métricas de Ejecución y Rendimiento

La siguiente tabla resume los resultados empíricos observados durante las pruebas automatizadas del pipeline de generación y despliegue:

Fase del Ciclo de Vida	Tiempo	Tokens LLM	Resultado y Comportamiento
Extracción de Requisitos (Spec)	1.4 s	1,250	Extracción estructurada de entidades y reglas BDD.
Scaffolding de Arquetipo	0.8 s	**0**	12 archivos generados limpiamente vía Jinja2.
Compilación Hermética Maven	4.2 s	0	Verificación Zero-Shot en sandbox (`BUILD SUCCESS`).
Construcción de Contenedor	12.1 s	0	Empaquetado OCI con `ubi8/openjdk-17-runtime`.
Despliegue OpenShip Local	3.5 s	0	Contenedores vinculados en red `openship_net`.
Verificación Health Check	0.05 s	0	Endpoint `/q/health` responde HTTP 200 (`STATUS UP`).

Table 6.1: Métricas Empíricas del Ciclo de Generación y Despliegue en AgentIA.

6.2 Demostración en Video por Hardware

Como prueba de ejecución completa en vivo sobre entorno Linux con Wayland, se grabó una sesión de despliegue mediante aceleración por GPU (VAAPI):

- **Archivo de Video:** `/home/gjanampa/Videos/quarkus_deployment_openship_demo.mp4`
- **Copia en Artefactos:** `brain/d48f01bf-708d-4c0f-8de9-a41c716e132b/quarkus_deployment_openshi`
- **Contenido Registrado:** Creación del proyecto, compilación Maven, arranque de OpenShip con PostgreSQL y verificación de endpoints REST activos.

6.3 Equivalencia para Despliegue en Microsoft Azure

El artefacto resultante cumple con los estándares exigidos para el despliegue directo en la nube pública:

1. **Azure Container Apps (ACA):** La imagen OCI generada se publica directamente en Azure Container Registry (ACR) y se ejecuta con escalado a cero (KEDA).

2. **Azure Kubernetes Service (AKS):** Los probes de salud (`/q/health/live` y `/q/health/ready`) se mapean directamente como `livenessProbe` y `readinessProbe`.

3. **Azure Database for PostgreSQL:** Basta con sobrescribir las variables de entorno `QUARKUS_DATASOURCE_J` y credenciales mediante Azure Key Vault.

Conclusiones

La arquitectura de AgentIA demuestra que la generación confiable de microservicios empresariales no depende de modelos masivos reescribiendo código boilerplate, sino de una **orquestación rigurosa**:

1. **Determinismo en la Base:** Scaffolding estructurado con arquetipos Quarkus 3.15 LTS que garantizan compilación al primer intento.
2. **Inteligencia en los Extremos:** LLM enfocado en la comprensión semántica inicial y en reparaciones quirúrgicas acotadas por P-Stack ante fallos del compilador.
3. **Observabilidad Total:** Transparencia visual mediante LangGraph Visualizer, streaming de terminal y la nube local OpenShip como paso previo a producción.